2026

Interactive Employee Directory - Project Documentation



Anjan Basak
Sumerix Global
10/28/2026

Interactive Employee Directory

Web Application - Complete Project Documentation

Advanced JavaScript (ES6+), Fetch API, REST APIs, JSON, Browser Storage, NPM and Vite

Internship	Sumerix Global - Full Stack Web Development Internship
Task	Task 2 of 6 - Advanced JavaScript and Interactive Employee Directory Web Application Development
Prepared by	Anjan Basak
Intern ID / Roll No.	_____
Mentor / Supervisor	_____
Submission date	30/10/2026
GitHub repository	https://github.com/123anjan/SumerixGlobal-projects-Task-2.git
Live demo (optional)	https://123anjan.github.io/SumerixGlobal-projects-Task-2/

Table of Contents

1. Introduction 3
2. Objectives and Learning Outcomes 3
3. Requirements Analysis 4
4. Tools and Technologies 4
5. System Design 5
6. Data Design 7
7. Implementation 8
8. Core JavaScript Concepts Applied 11
9. Browser Storage 13
10. User Interface, Accessibility and Responsiveness 14
11. Application Screenshots 15
12. Testing and Debugging 18
13. Errors Encountered and Troubleshooting 19
14. Performance Optimisation 20
15. Development Process 20
16. Deployment 21
17. Evaluation Criteria Mapping 22
18. Final Submission Checklist 22
19. Challenges and Key Learnings 23
20. Future Enhancements and Connection to Task 3 23
21. Conclusion 24
22. References and Resources 24
Appendix A. Source Code Listings 26
Appendix B. Interview Preparation 28

1. Introduction

The Interactive Employee Directory is a single-page web application that displays a company's employees in a searchable, filterable and sortable list. When a user selects an employee, a side panel shows contact, work and personal information. Users can mark colleagues as favorites, switch between light and dark mode, and the application remembers these choices the next time it is opened.

The project was built as **Task 2** of the Sumerix Global Full Stack Web Development Internship. It moves from static HTML and CSS (Task 1) to dynamic, data-driven web development: the application retrieves employee information from a public REST API using the Fetch API, handles asynchronous operations with Promises and async/await, parses JSON data, stores preferences in the browser, and is organised into reusable ES modules built and served with Vite.

1.1 Project at a glance

Item	Details
Project title	Interactive Employee Directory Web Application
Type	Front-end single-page application (no custom backend)
Language	JavaScript (ES6+ modules), HTML5, CSS (Tailwind CSS)
Build tool	Vite with the npm package manager
Data source	RandomUser public REST API (JSON)
Storage	Local Storage (preferences) and Session Storage (tab data and API cache)
Key features	List, search, department filter, availability filter, sorting, favorites, detail panel, dark mode, loading / empty / error states, responsive and accessible UI
Estimated duration	30-35 days (learning plan included in Section 15)

1.2 Why this project matters

Modern web applications depend on JavaScript for interactivity and for communicating with back-end services. A professional front-end developer must understand ES6+ syntax, asynchronous programming, REST APIs, JSON, browser storage, package management and development tooling. This project combines all of these in one practical application, and it provides the foundation for React.js (Task 3) and for the back-end integration in the later tasks of the internship capstone.

1.3 Scope

- **In scope:** front-end application, API integration, client-side filtering and sorting, browser storage, responsive and accessible UI, documentation and a GitHub repository.
- **Out of scope:** custom back-end, database, user authentication and editing employee records (these are planned for later internship tasks).

2. Objectives and Learning Outcomes

2.1 Objectives

- Strengthen JavaScript skills using modern ES6+ features.
- Understand asynchronous programming, the event loop, Promises and async/await.
- Communicate with a REST API using the Fetch API and handle errors correctly.
- Parse and manage JSON data (`JSON.parse()` and `JSON.stringify()`).

- Persist user preferences using Local Storage and Session Storage.
- Use Node.js, npm and Vite to create a professional front-end project.
- Organise a scalable application into small modules and reusable components.
- Build a responsive and accessible user interface.
- Publish the project on GitHub with professional documentation.

2.2 Learning outcomes

After completing this task the learner can build interactive web applications using modern JavaScript, consume REST APIs with the Fetch API, handle asynchronous code with Promises and async/await, manage browser data with Local Storage and Session Storage, create projects with Node.js, npm and Vite, and organise scalable JavaScript applications using modules and reusable components.

3. Requirements Analysis

3.1 Functional requirements

ID	Requirement	Priority
FR-01	Display a list of employees (photo, name, job title, department, phone, availability).	Must
FR-02	Retrieve employee information from a REST API using fetch().	Must
FR-03	Search employees by name (case-insensitive, debounced).	Must
FR-04	Filter by department.	Must
FR-05	Filter by availability (Available / Out).	Should
FR-06	Sort employee records (A-Z, Z-A, by department).	Must
FR-07	Show detailed information of the selected employee.	Must
FR-08	Save favorite employees and allow viewing favorites only.	Must
FR-09	Store preferences locally (favorites, theme, department, sort).	Must
FR-10	Show loading, empty and error states, with a Retry button on errors.	Must
FR-11	Provide a dark mode that is remembered.	Should
FR-12	Work on desktop, tablet and mobile devices.	Must

3.2 Non-functional requirements

- **Accessibility:** labelled controls, keyboard operation, visible focus, ARIA attributes.
- **Performance:** debounced search, cached API response, lazy-loaded images, small production build.
- **Maintainability:** modular folder structure, single-purpose files, readable code.
- **Security:** all API text is escaped before it is inserted into the page (prevents HTML injection).
- **Compatibility:** works in current versions of Google Chrome and Mozilla Firefox.
- **Reliability:** the app keeps working if browser storage is blocked, and shows a friendly message if the API fails.

4. Tools and Technologies

Category	Tool	Purpose
Editor	Visual Studio Code	Writing code (ESLint, Prettier, Live Server, GitLens are useful extensions)
Browsers	Google Chrome, Mozilla Firefox	Running, debugging and cross-browser testing
Runtime	Node.js (LTS) and npm	Runs Vite and installs packages
Build tool	Vite	Development server with hot reload, production build, project structure
Styling	Tailwind CSS (v4)	Utility-first styling, responsive breakpoints, dark mode variant
Language	JavaScript ES6+ modules	Application logic
API testing	Postman	Testing GET requests and inspecting JSON before coding
Version control	Git and GitHub	Source control and publishing
Data source	RandomUser API	Free public API returning realistic user profiles
Deployment (optional)	Vercel / Netlify / GitHub Pages	Hosting the finished build

5. System Design

5.1 Architecture overview

The application follows a simple layered architecture. Each layer has exactly one responsibility, and data flows in one direction: from the API, through the service layer, into a central state object, and finally into the components that draw the interface. Because the API, service, storage and utility layers contain no DOM code, they can be reused unchanged when the project is migrated to React in Task 3.

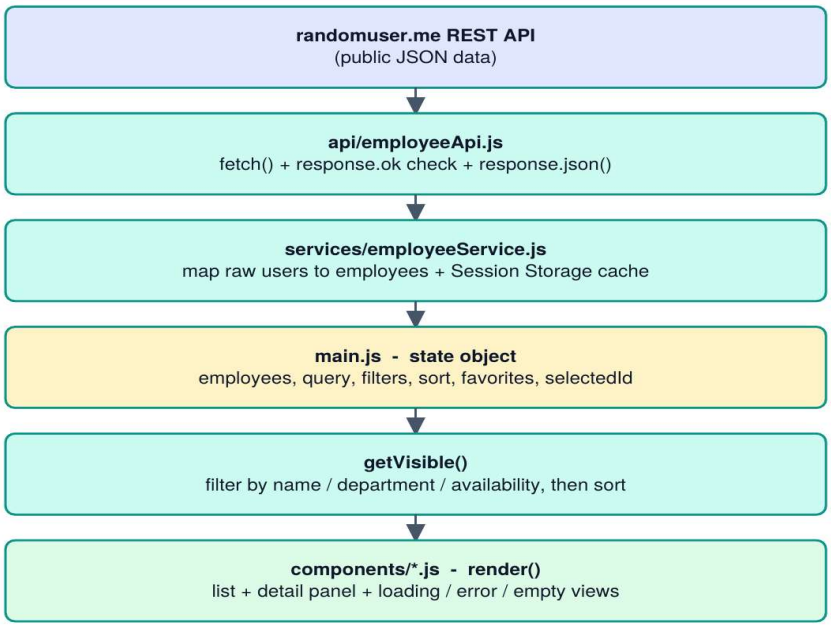


Figure 1: Data flow of the application (API to screen)

5.2 Module responsibilities

File / folder	Responsibility
index.html	Page skeleton: header, filter form, list container, detail panel, script tag (type=module).
src/main.js	Entry point. Holds the state object, wires events, runs getVisible() and render(), handles load().
src/api/employeeApi.js	Calls the REST API with fetch(), checks response.ok and returns the parsed JSON results.
src/services/employeeService.js	Converts raw API users into employee objects, adds department, title, availability and supervisor, and caches the result in Session Storage.
src/storage/storage.js	Safe wrappers (get / set / remove) around Local and Session Storage using JSON and try/catch.
src/components/employeeList.js	Builds the HTML for the list of employees (with favorite stars).
src/components/detailPanel.js	Builds the HTML for the selected employee's contact, work and personal information.
src/components/stateViews.js	Loading spinner, error screen with Retry, and empty-result view.
src/utils/helpers.js	debounce() and escapeHtml() helper functions.
src/js/constants.js	Department names, job titles and storage keys.
src/css/style.css	Imports Tailwind and defines the dark-mode variant.
vite.config.js	Vite configuration and the Tailwind plugin.

5.3 Folder structure

```
Task-2/
|-- public/                     static files copied as-is
|-- src/
|   |-- api/          employeeApi.js
|   |-- assets/       images and static assets
|   |-- components/   employeeList.js, detailPanel.js, stateViews.js
|   |-- css/          style.css
|   |-- js/           constants.js
|   |-- services/     employeeService.js
|   |-- storage/      storage.js
|   |-- utils/        helpers.js
|   `-- main.js
|-- index.html
|-- package.json
|-- vite.config.js
|-- README.md
`-- .gitignore
```

5.4 User interface layout

The interface follows the reference design supplied with the task: a header, a filter bar, a table-style list of employees on the left, and a details panel on the right. On small screens the table columns collapse and the details panel moves below the list.

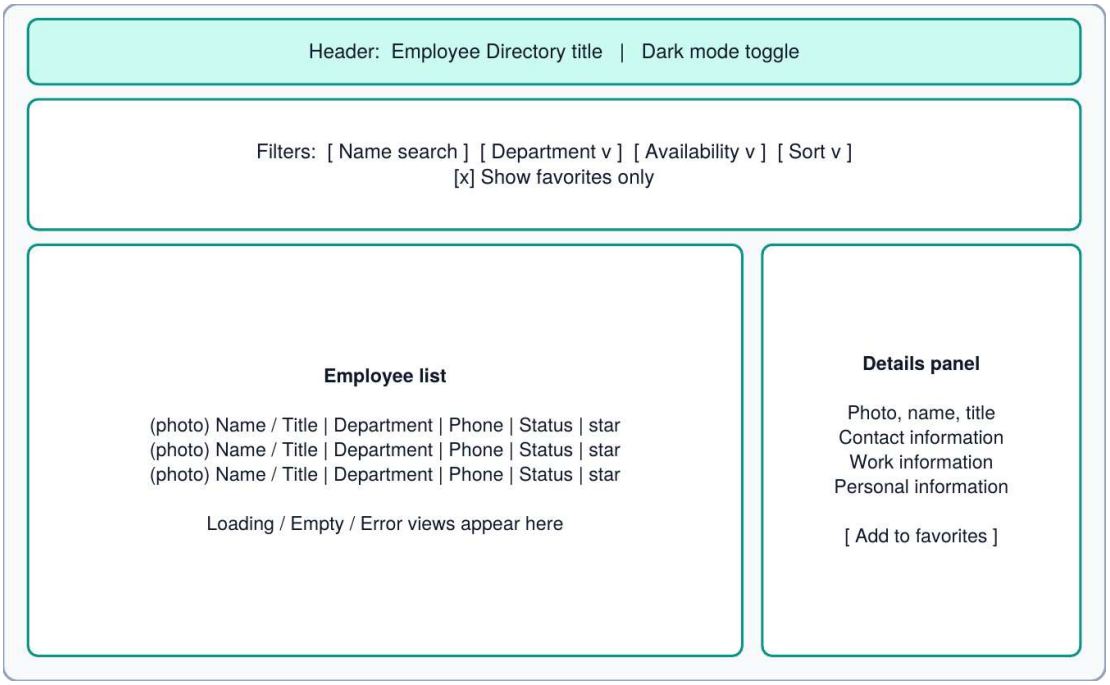


Figure 2: Wireframe of the Employee Directory layout

6. Data Design

6.1 REST API

The application uses the free **RandomUser API**. A single GET request returns 30 realistic users as JSON. The `seed` parameter makes the API return the same people every time, which keeps the directory stable between page loads.

```
GET https://randomuser.me/api/?results=30&seed=sumerix&nat=us,gb,au
    &inc=login,name,email,phone,cell,picture,location,dob,gender
```

Query parameter	Meaning
results=30	Number of users to return
seed=sumerix	Deterministic results (same people on every request)
nat=us,gb,au	Nationalities to include
inc=...	Return only the listed fields (smaller response)

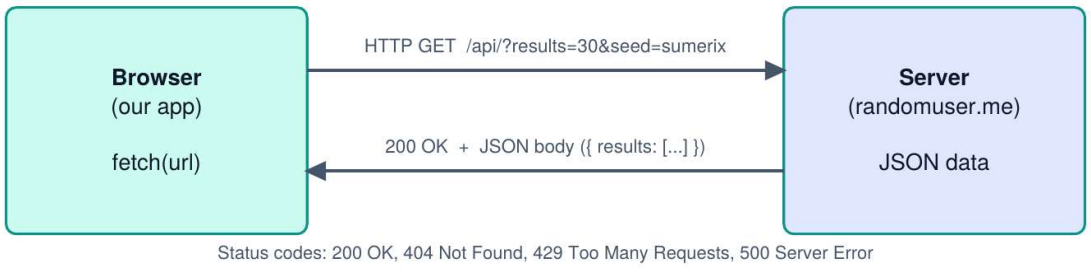


Figure 3: REST API request / response cycle

6.2 Sample JSON response (simplified)

```
{
  "results": [
    {
      "login":    { "uuid": "7a0f..." },
      "name":     { "first": "Cole", "last": "Ashbury" },
      "email":    "cole@example.com",
      "phone":    "(728) 521-2584",
      "cell":     "(472) 276-0193",
      "picture":  { "medium": "https://randomuser.me/api/portraits/med/men/1.jpg" },
      "location": { "city": "Atlanta" },
      "dob":      { "date": "1990-09-29T00:00:00.000Z" },
      "gender":   "male"
    }
  ]
}
```

6.3 Employee model

The service layer converts every raw API user into a simpler employee object used by the interface. Fields marked *derived* are not provided by the API; they are generated in a predictable way so the directory has the departments and availability that the design requires.

Field	Source	Description
id	login.uuid	Unique identifier used for favorites and selection
name	name.first + name.last	Full name
title	derived	Job title chosen from the department's title list
department	derived	One of Engineering, Accounting, Sales, Marketing, Finance, Design (assigned by index)
availability	derived	'Available' or 'Out' (every fifth employee is Out)
phone / mobile	phone / cell	Office telephone and mobile number
email	email	Email address
photo	picture.medium	Profile picture URL
office / city	location.city	City shown in work and personal information
gender	gender	Shown as Male / Female
birthday	dob.date	Formatted as day and month
supervisor	derived	Another employee from the same department

6.4 Application state

```
const state = {
  employees:     [],                // data from the service
  query:         '',                // search text
  department:    'All',             // restored from Local Storage
  availability:  'All',
  sort:          'az',              // restored from Local Storage
  favoritesOnly: false,
  favorites:     new Set([...]),    // restored from Local Storage
  selectedId:    null               // restored from Session Storage
};
```

7. Implementation

7.1 Environment setup (Steps 1-2 of the brief)

Node.js (LTS) and npm were installed and verified with `node -v` and `npm -v`. The project was created with Vite's vanilla JavaScript template and Tailwind CSS was added as a dependency.

```
cd "SumerixGlobal projects"
npm create vite@latest Task-2 -- --template vanilla
cd Task-2
npm install
npm install tailwindcss @tailwindcss/vite
npm run dev          # development server on http://localhost:5173
```

vite.config.js registers the Tailwind plugin, and **src/css/style.css** imports Tailwind and enables a class-based dark mode:

```
// vite.config.js
import { defineConfig } from 'vite';
import tailwindcss from '@tailwindcss/vite';
export default defineConfig({ plugins: [tailwindcss()] });

/* src/css/style.css */
@import "tailwindcss";
@custom-variant dark (&:where(.dark, .dark *));
```

7.2 API layer

`employeeApi.js` builds the URL with `URLSearchParams`, calls `fetch()`, and checks `response.ok`. Important: fetch() only rejects on network failure; an HTTP 404 or 500 still resolves. Therefore the code throws its own error when the status is not successful, so that the interface can show an error screen.

```
export async function fetchUsers(count = 30) {
  const params = new URLSearchParams({ results: count, seed: 'sumerix',
    nat: 'us,gb,au', inc: 'login,name,email,phone,cell,picture,location,dob,gender' });
  const response = await fetch(`https://randomuser.me/api/?${params}`);
  if (!response.ok) throw new Error(`Request failed (HTTP ${response.status})`);
  const data = await response.json();
  return data.results;
}
```

7.3 Service layer and caching

`employeeService.js` first looks for cached data in Session Storage. If none exists it calls the API, maps each user to an employee (adding the derived fields), assigns supervisors, saves the result to Session Storage and returns it. This optimises API requests (exercise 12): reloading the page in the same tab does not call the network again.

7.4 Storage layer

`storage.js` exports two objects, `local` and `session`, created by one factory function. Every value is saved with `JSON.stringify()` and read with `JSON.parse()`. All calls are wrapped in `try/catch`, so private mode, blocked storage or corrupted values never crash the application; a default value is returned instead.

7.5 Filtering, searching and sorting

The function `getVisible()` applies all active conditions in a single pass using `Array.filter()` and then sorts a copy of the result with `Array.sort()` and `localeCompare()`.

```
function getVisible() {
  const q = state.query.trim().toLowerCase();
  const result = state.employees.filter((e) =>
    e.name.toLowerCase().includes(q) &&
    (state.department === 'All' || e.department === state.department) &&
    (state.availability === 'All' || e.availability === state.availability) &&
    (!state.favoritesOnly || state.favorites.has(e.id)));
  const sorters = {
    az:   (a, b) => a.name.localeCompare(b.name),
    za:   (a, b) => b.name.localeCompare(a.name),
    dept: (a, b) => a.department.localeCompare(b.department) || a.name.localeCompare(b.name) };
  return result.sort(sorters[state.sort]);
}
```

The search box uses **debounce** (300 ms). The handler waits until the user pauses typing, so the list is re-rendered once instead of on every keystroke. This is an example of a closure: the returned function remembers the `timer` variable.

7.6 Rendering and events

The application uses a simple *state in, UI out* approach. Any event updates the state object and then calls `render()`, which asks `getVisible()` for the list, draws the list or the empty view, updates the result counter, and redraws the details panel. Clicks on rows, stars and the Retry button are handled by **event delegation**: one listener on the list container checks `event.target.closest()` for `data-select`, `data-fav` or the retry button. This works even though the rows are re-created on every render.

```
async function load() {
  showLoading(els.list);                  // 1. loading state
  try {
    state.employees = await getEmployees();   // 2. wait for API / cache
    render();                                  // 3. success (or empty) state
  } catch (error) {
    showError(els.list, error.message);    // 4. error state with Retry button
  }
}
```

7.7 Favorites

Favorites are kept in a JavaScript `Set` of employee IDs, which makes adding, removing and testing membership fast. Whenever a star is clicked the set is updated, saved to Local Storage as an array (`[...state.favorites]`) and the interface is re-rendered. On startup the set is rebuilt from storage.

7.8 Dark mode

The theme button toggles the `dark` class on the `<html>` element, which activates Tailwind's `dark:` styles. The choice is saved in Local Storage. On the first visit the app follows the operating system preference using `window.matchMedia('(prefers-color-scheme: dark)')`.

7.9 Loading, empty and error states

State	Trigger	What the user sees	Accessibility
Loading	Waiting for the API	Spinner and 'Loading employees...'	role='status'
Success	Data received	Employee list and result count	aria-live='polite' count
Empty	No employee matches	'No employees found' with a hint	Plain text message
Error	Network or HTTP failure	Error message and Retry button	role='alert'

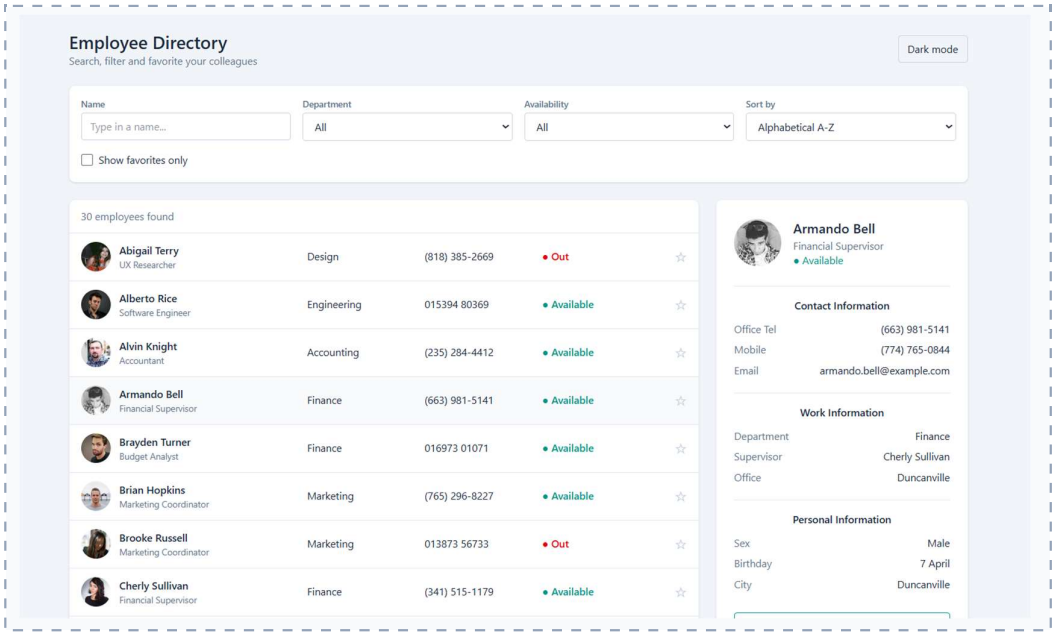


Figure 4: Main screen of the Employee Directory (light mode)

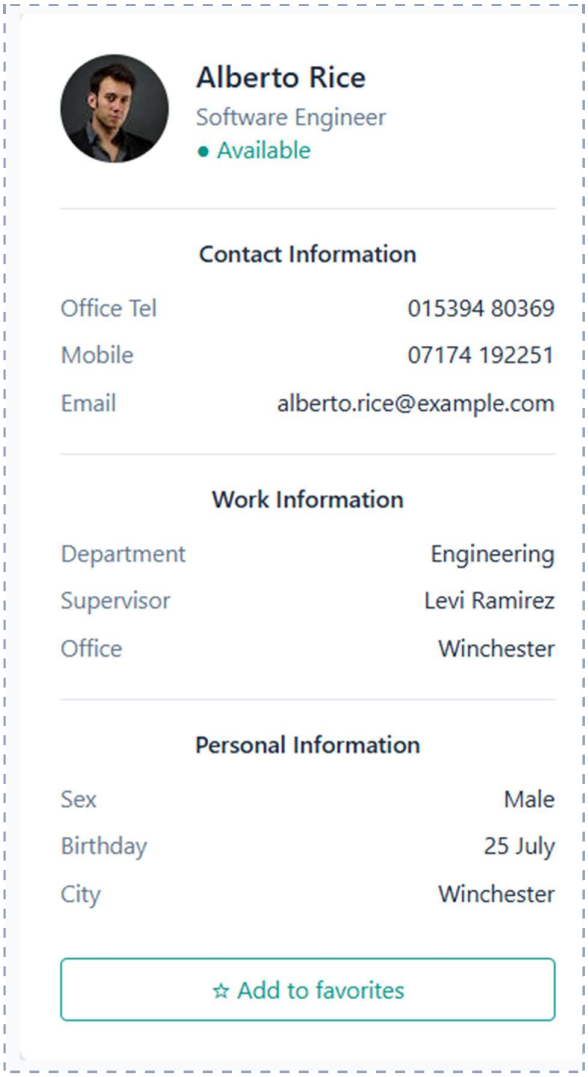


Figure 5: Details panel for a selected employee

8. Core JavaScript Concepts Applied

8.1 ES6+ features used

Concept	Where it is used
let / const	All variable declarations (const by default)
Arrow functions	Event handlers, array callbacks, sorters
Template literals	Building HTML strings and URLs
Destructuring and spread	Function parameters, [...state.favorites], copying data
Default and rest parameters	fetchUsers(count = 30), debounce(fn, delay = 300), (...args)
Optional chaining / nullish coalescing	local.get(theme) ?? system preference, boss ? boss.name : '-'
Modules	import / export in every file; type='module' script
Classes	Practised in exercises; the app uses functions and factories
Higher-order functions	debounce, map, filter, sort, find, forEach
Closures	debounce (timer), makeStore (area)
Set	Favorites collection

8.2 Asynchronous JavaScript

JavaScript runs on a single thread. Long operations such as network requests are handed to the browser's Web APIs so the page stays responsive. When the work finishes, its callback waits in a queue. The **event loop** moves it to the call stack only when the stack is empty. Promise callbacks (`.then`, code after `await`) go to the higher-priority **microtask queue**.

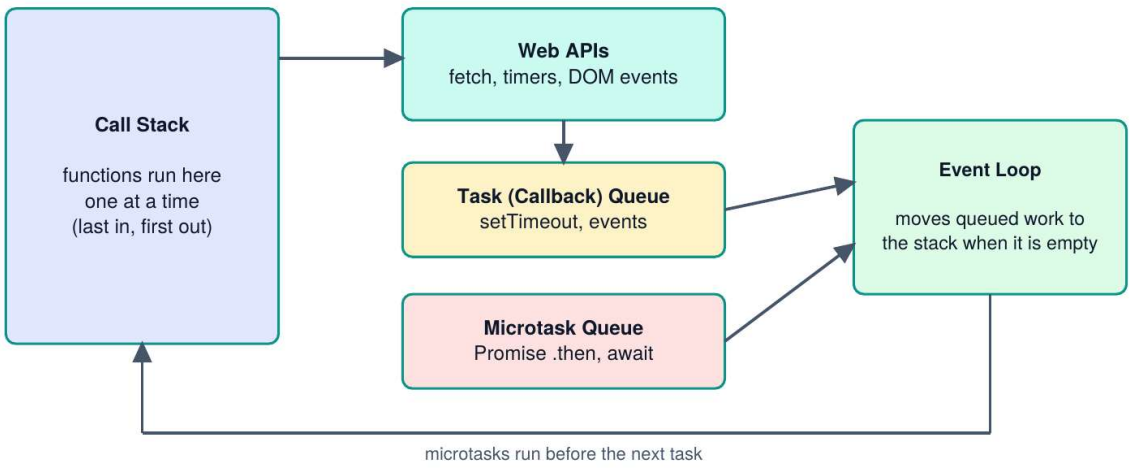


Figure 6: Call stack, Web APIs, callback queue, microtask queue and event loop

A **Promise** represents a value that will be available later. It starts *pending* and ends either *fulfilled* (success) or *rejected* (failure). `async/await` is syntax that lets asynchronous code read like synchronous code, and errors are handled with `try/catch`.

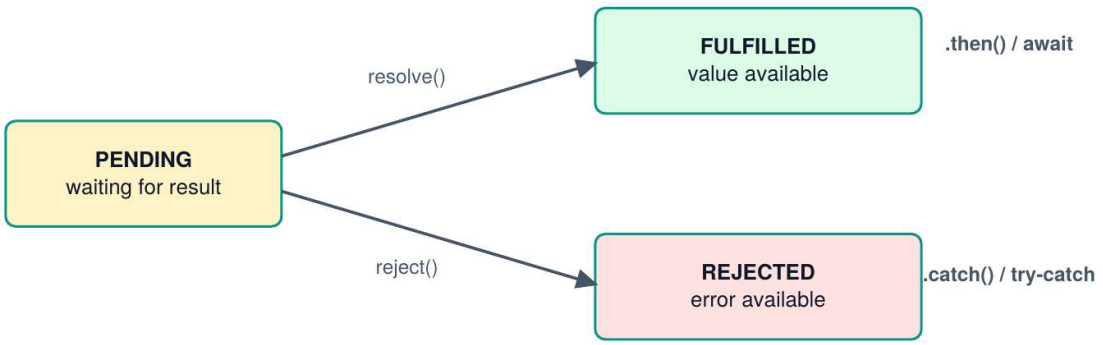


Figure 7: Promise lifecycle

```
// Same logic written two ways
fetch(url).then(r => r.json()).then(show).catch(showError);     // Promise chain

try { const r = await fetch(url); show(await r.json()); }        // async / await
catch (err) { showError(err); }
```

8.3 Fetch API methods

HTTP method	Purpose	Fetch example
GET	Read data	fetch(url)
POST	Create data	fetch(url, { method: 'POST', headers: {...}, body: JSON.stringify(data) })
PUT	Replace data	fetch(url, { method: 'PUT', ... })
PATCH	Update part of data	fetch(url, { method: 'PATCH', ... })
DELETE	Remove data	fetch(url, { method: 'DELETE' })

This application only needs GET. The other methods were practised with Postman and JSONPlaceholder / ReqRes and will be used when a real back-end is added.

8.4 JSON

JSON (JavaScript Object Notation) is the text format used to exchange data with APIs. `response.json()` parses the response body into JavaScript objects and arrays. `JSON.stringify()` turns data into a string for storage (serialization) and `JSON.parse()` turns it back (deserialization).

9. Browser Storage

Web Storage lets the browser keep small amounts of key/value data. Local Storage persists until it is cleared; Session Storage lives only as long as the browser tab. Both store strings only, so objects are converted with JSON.

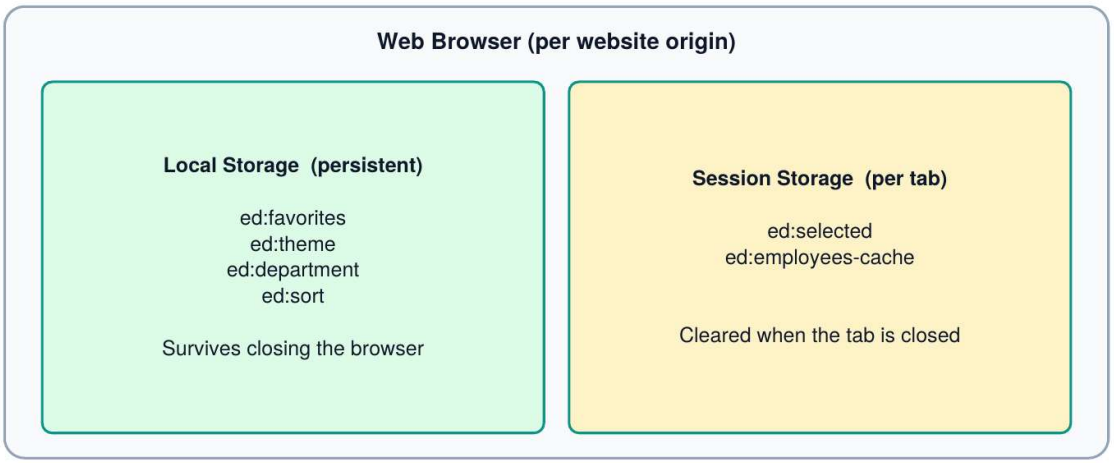


Figure 8: Local Storage and Session Storage used by the application

Key	Type	Value	Purpose
ed:favorites	Local	Array of employee IDs	Remember favorites
ed:theme	Local	'light' or 'dark'	Remember dark mode
ed:department	Local	Department name	Restore last filter
ed:sort	Local	'az', 'za' or 'dept'	Restore last sort
ed:selected	Session	Employee ID	Keep selection on reload in the tab
ed:employees-cache	Session	Array of employees	Avoid repeated API calls

Cookies were studied as an introduction only. They are sent to the server with every request and are mainly used for authentication, so they are not used in this project. **Storage events** fire in other tabs when Local Storage changes and could be used in future to synchronise the favorites between tabs.

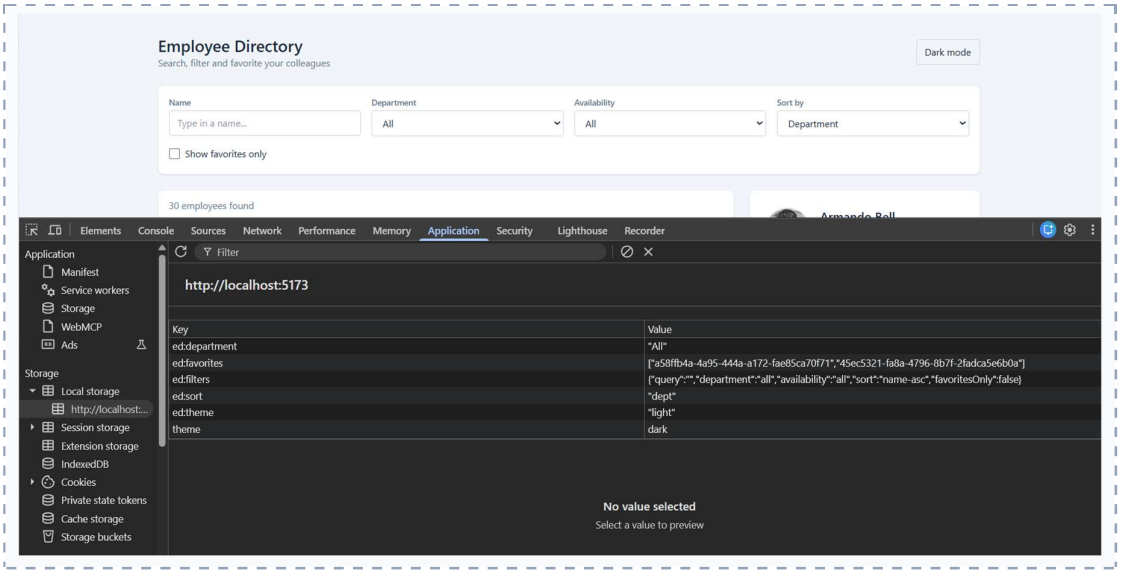


Figure 9: Local Storage entries inside Chrome DevTools

10. User Interface, Accessibility and Responsiveness

10.1 Design approach

The layout was inspired by the reference design provided with the task: a clean white interface with a teal accent colour, a table-style employee list with round profile photos, green 'Available' and red 'Out' status labels, and a right-hand details panel with three groups of information. Tailwind CSS utility classes were used for spacing, colours, typography and the responsive grid.

10.2 Responsive behaviour

Screen	Breakpoint	Layout
Mobile	below 640 px	Single column. List shows photo, name, title and status. Details panel appears below the list. Filters stack vertically.
Tablet	640 px and above	Two-column filter grid. List also shows department and phone columns.
Desktop	1024 px and above	Four-column filter bar. List on the left and details panel on the right (340 px).

10.3 Accessibility measures

- Every input and select has a visible `<label>`.
- The whole interface can be used with the keyboard (Tab, Shift+Tab, Enter, Space); rows and stars are real buttons.
- Visible focus outlines (`focus-visible`) on all interactive elements.
- `aria-live='polite'` announces the number of results after searching or filtering.
- `role='status'` on the loader and `role='alert'` on errors.
- `aria-pressed` on the favorite and dark-mode buttons; descriptive `aria-label` on each star.
- Decorative images use empty alt text; the large profile photo has a descriptive alt.
- The page language is declared (`lang='en'`) and the colour contrast is checked with Lighthouse.

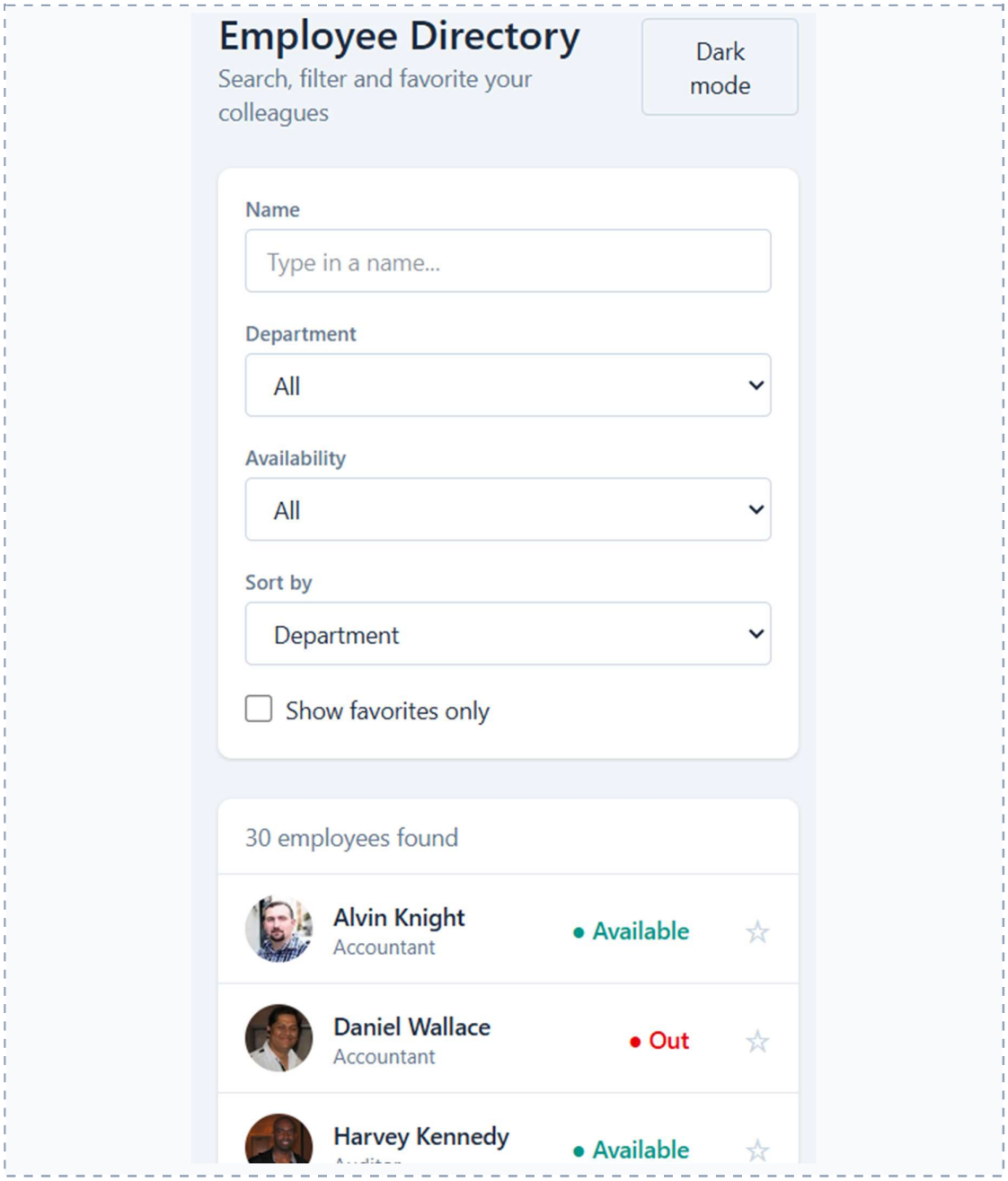


Figure 10: Mobile layout

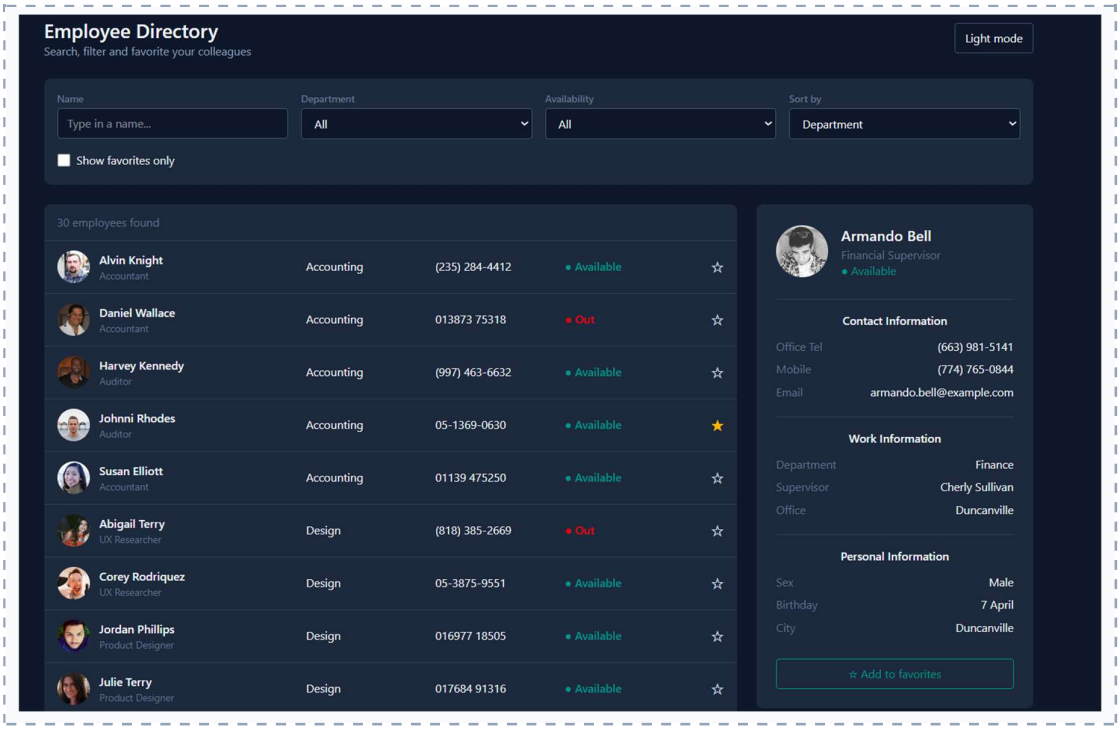


Figure 11: Dark mode

11. Application Screenshots

Paste the final screenshots of the running application in the frames below (select the frame, insert an image and resize it to fit, or replace this page in your word processor / PDF editor). The frames are grouped by the states the application can be in.

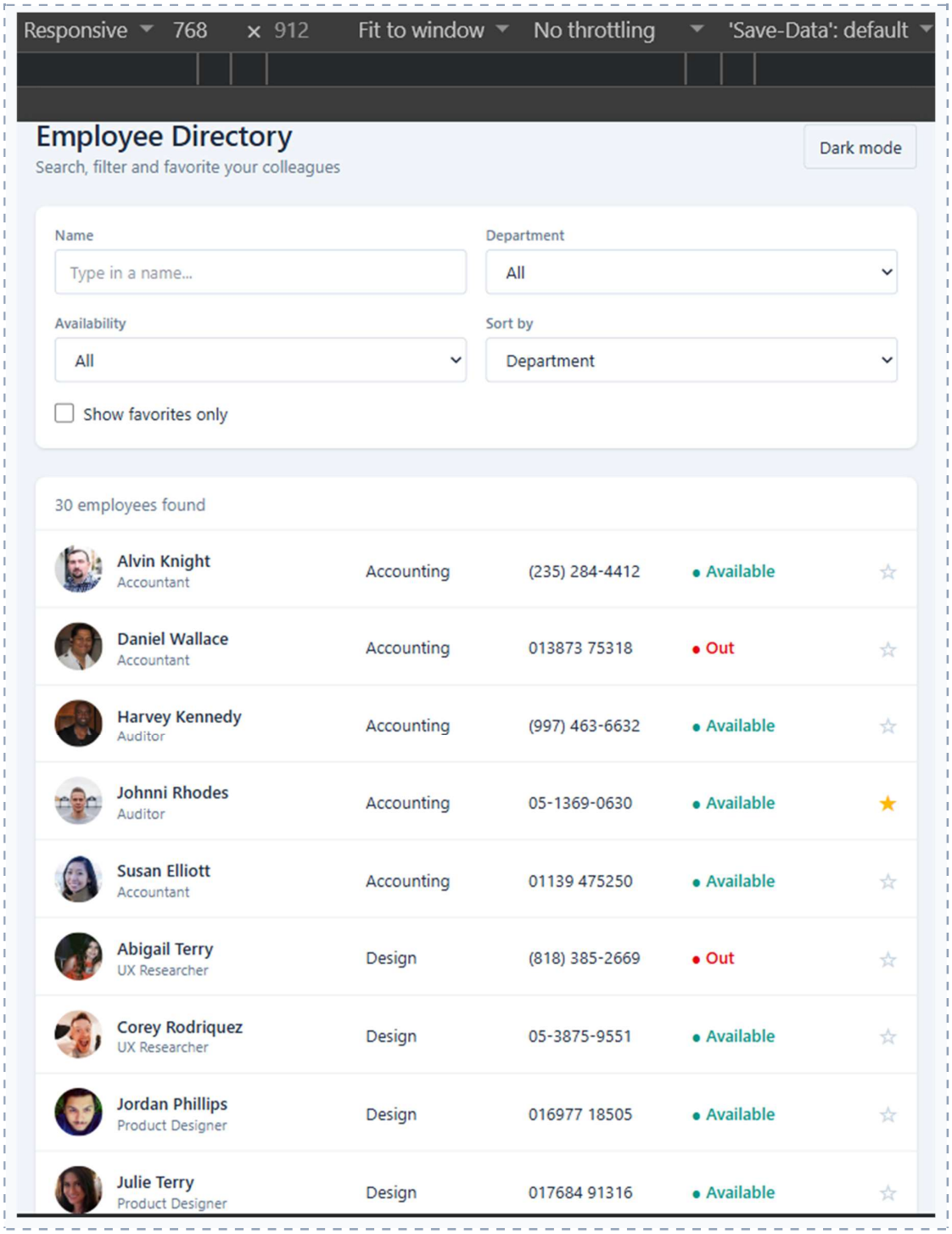


Figure 12: Tablet layout

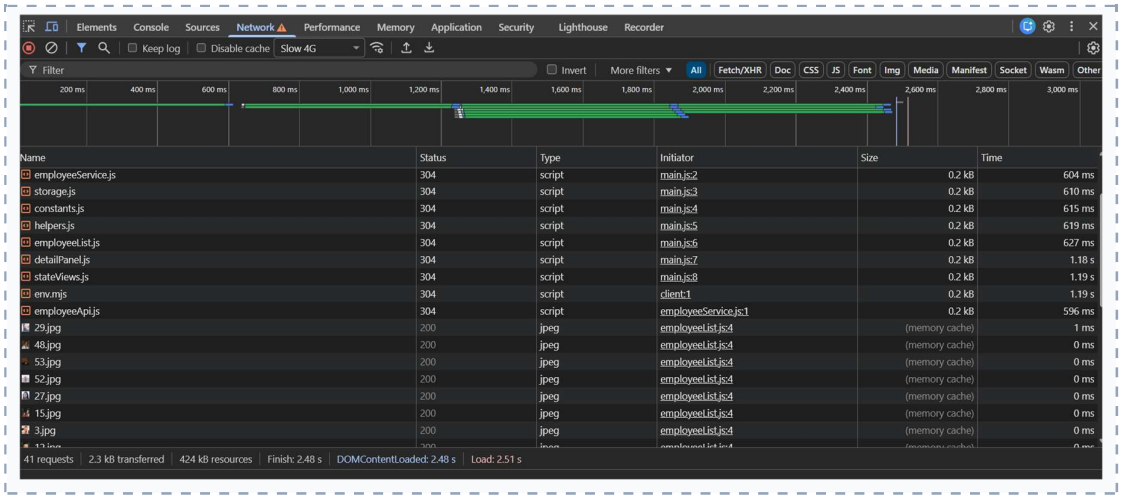


Figure 13: Loading state

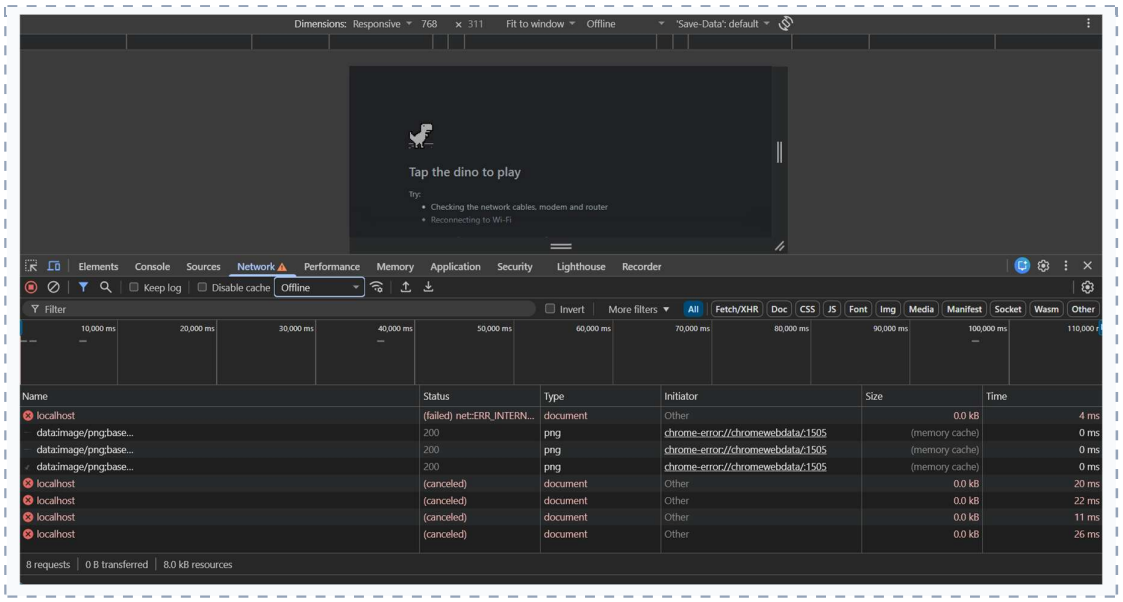


Figure 14: Error state

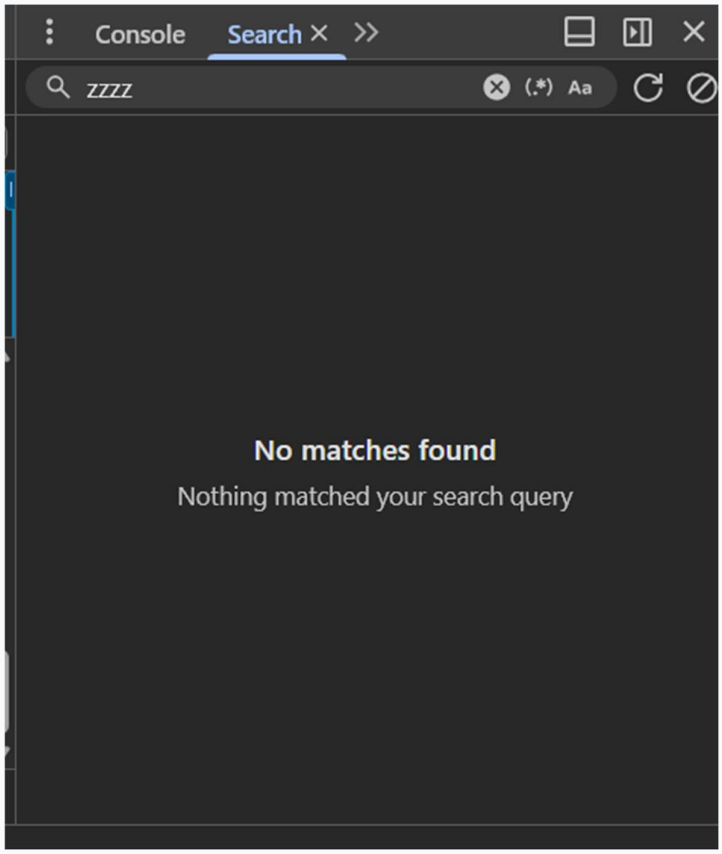


Figure 15: Empty state

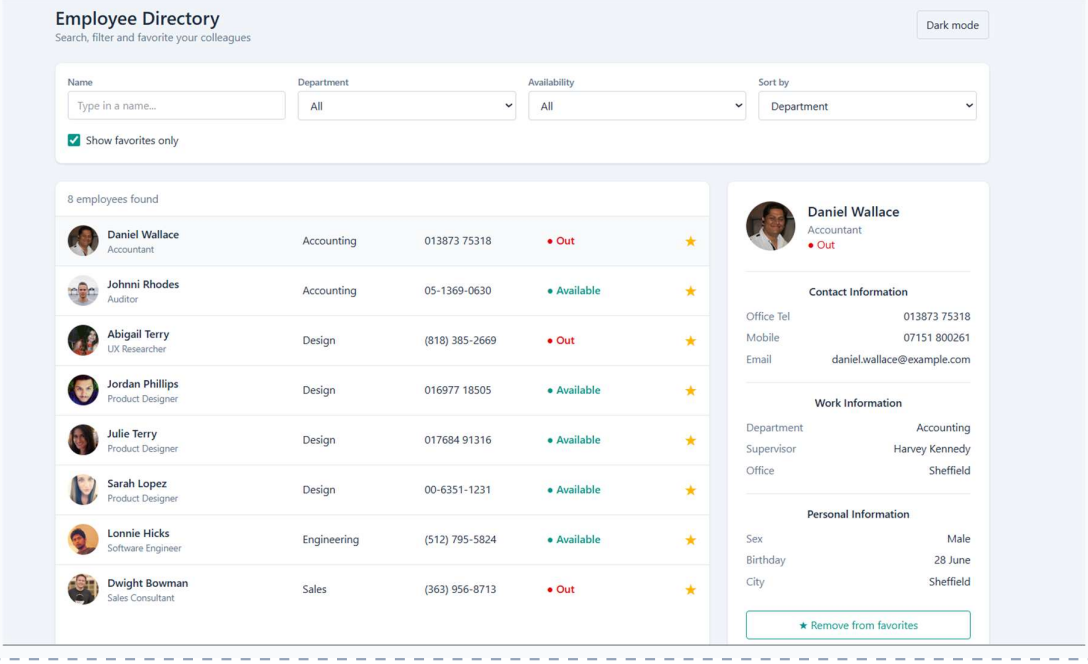


Figure 16: Favorites view

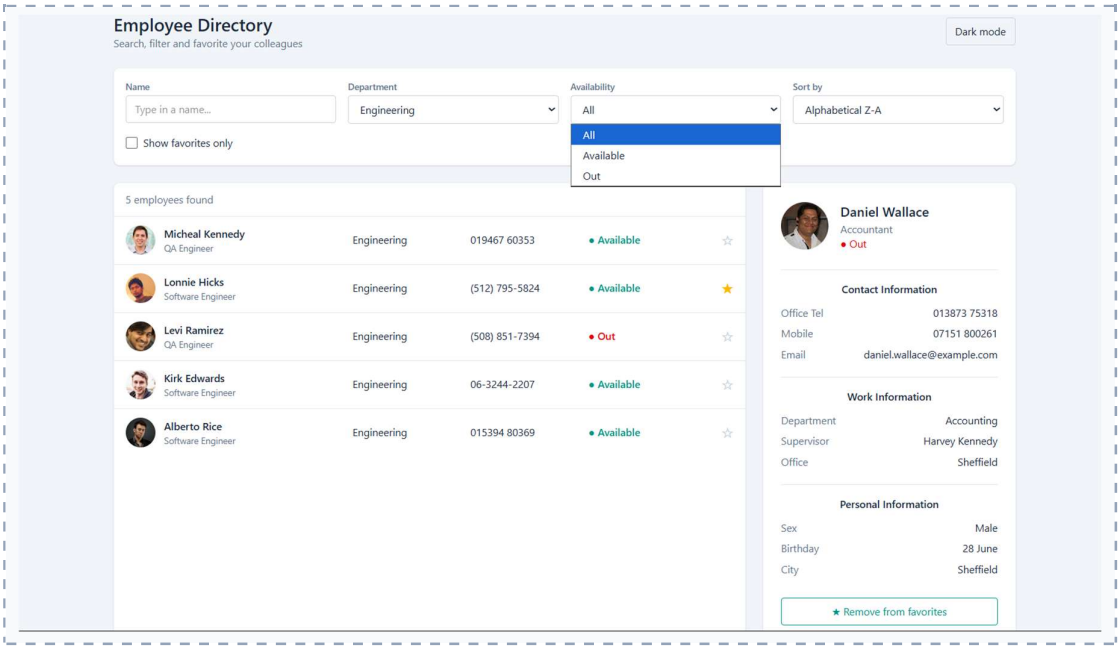


Figure 17: Filtering and sorting

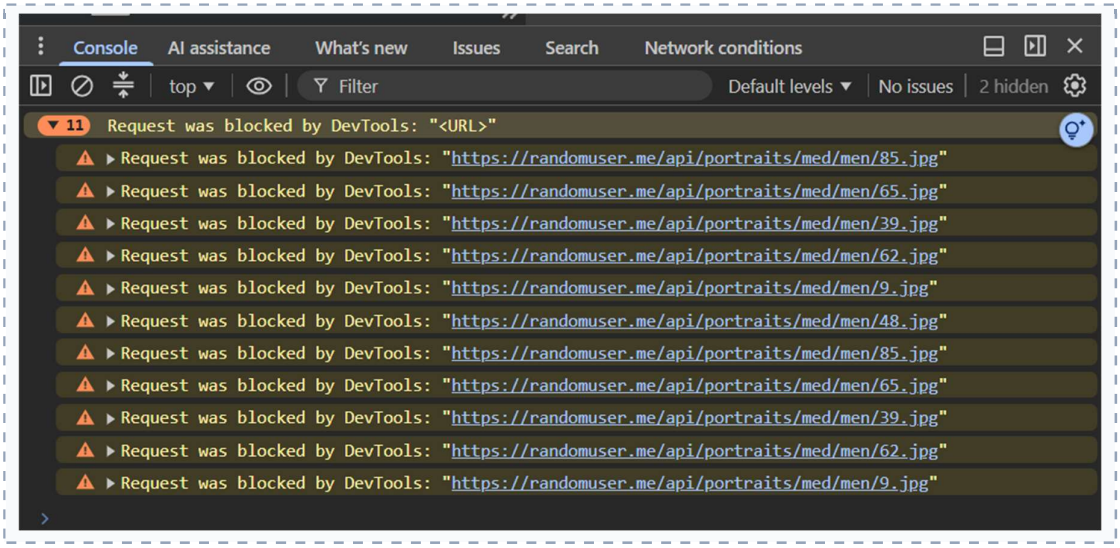


Figure 18: Network tab - API request

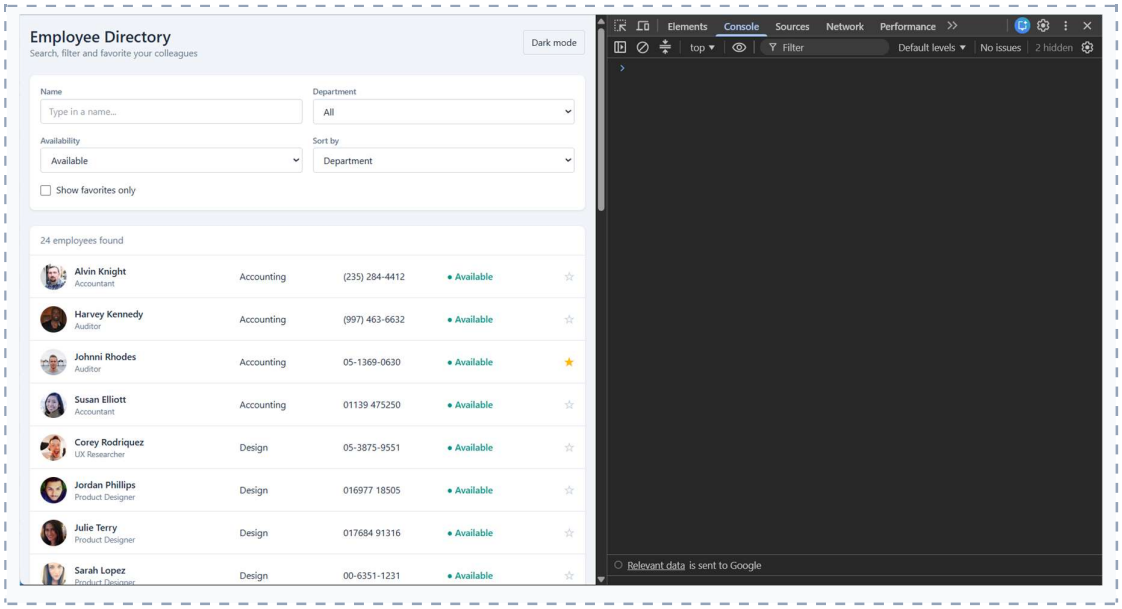


Figure 19: Console without errors

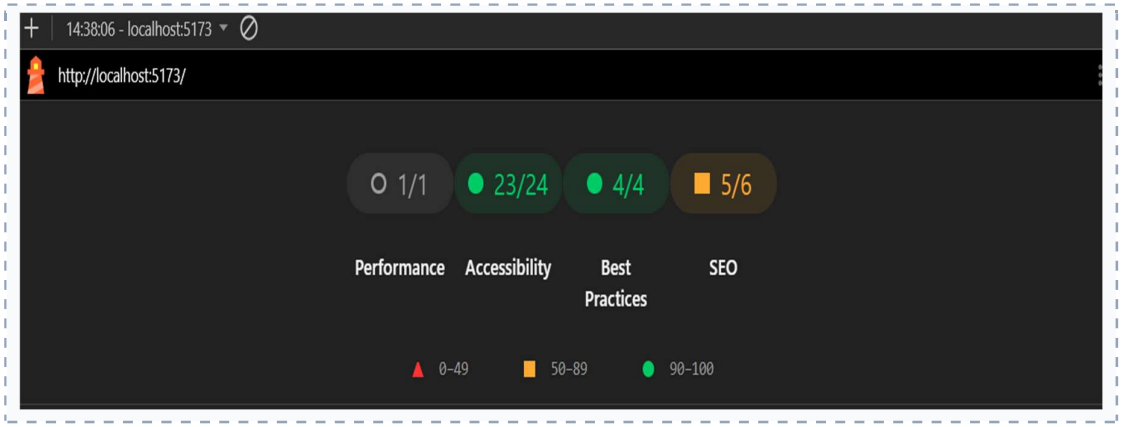


Figure 20: Lighthouse accessibility audit

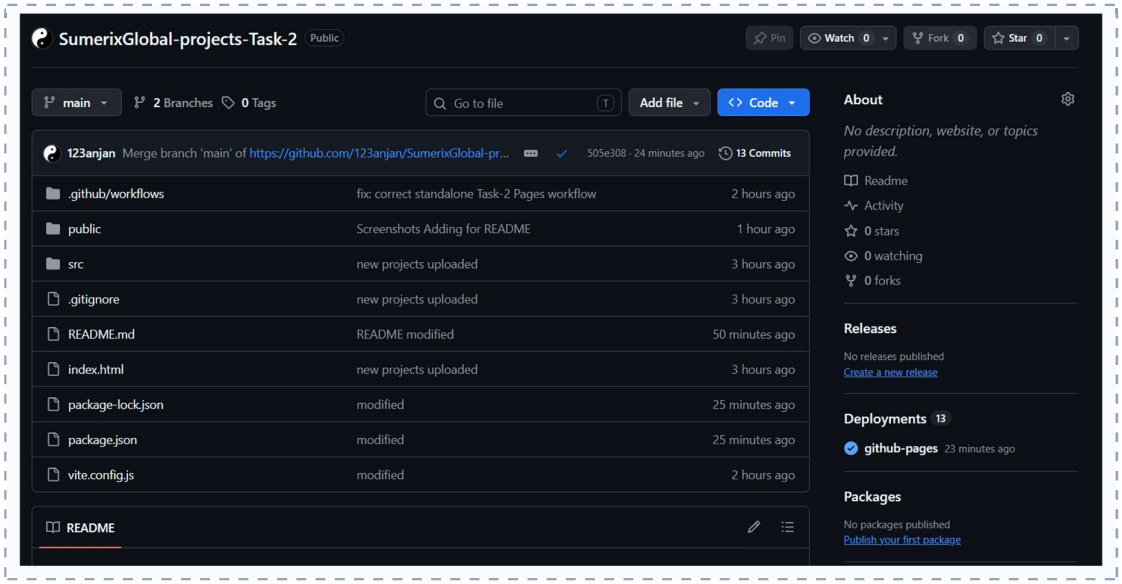


Figure 21: GitHub repository

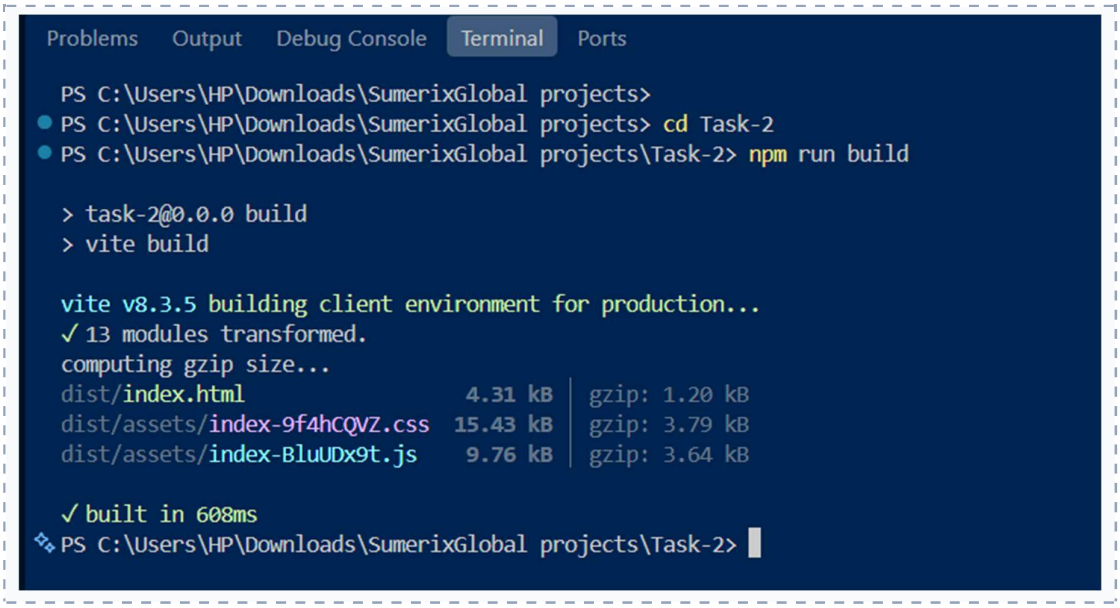


Figure 22: Production build

12. Testing and Debugging

12.1 Test plan

The application was tested manually in Chrome and Firefox at mobile, tablet and desktop widths. Write Pass or Fail in the Result column when you run each test.

ID	Test	Steps	Expected result	Result
T01	Initial load	Open the app	Spinner, then 30 employees	____
T02	Search	Type 'a' in Name	Only names containing 'a'; count updates	____
T03	Search - empty	Type 'zzzz'	'No employees found' view	____
T04	Department filter	Select Marketing	Only Marketing employees	____
T05	Availability filter	Select Out	Only employees marked Out	____
T06	Sort A-Z / Z-A	Change Sort by	List order reverses correctly	____
T07	Sort by department	Choose Department	Grouped by department, names A-Z inside	____
T08	Select employee	Click a row	Details panel shows that person	____
T09	Add favorite	Click the star	Star filled; button text changes	____
T10	Favorites only	Tick the checkbox	Only favorites are listed	____
T11	Persistence	Favorite, then reload	Favorites still selected	____
T12	Dark mode	Click the toggle, reload	Dark theme stays	____
T13	Offline error	Clear session cache, go Offline, reload	Error screen and Retry button	____
T14	Retry	Go Online, click Retry	List loads	____
T15	Keyboard only	Use Tab, Enter, Space	Every control reachable and usable	____
T16	Responsive	Resize to 375 px	No horizontal scrolling; panel below list	____
T17	Build	npm run build, npm run preview	Build succeeds; preview works	____

12.2 Debugging tools used

Tool	Used for
Console	Reading errors and warnings; console.log(state) inside render()
Breakpoints (Sources tab)	Pausing inside getVisible() and load() to inspect variables step by step
Network tab	Checking the request URL, status code, response JSON, timing and throttling
Application tab	Viewing and clearing Local Storage and Session Storage
Elements tab	Checking classes (dark mode) and ARIA attributes
Lighthouse	Accessibility, performance and best-practice audit
Postman	Testing the API URL before writing code

13. Errors Encountered and Troubleshooting

The table lists the errors most likely to appear while building and running the project, with the cause and the fix.

Error / symptom	Likely cause	Fix
'npm' is not recognized	Node.js not installed or PATH not updated	Install Node.js LTS, then close and reopen the terminal / VS Code.
Vite requires a newer Node.js version	Old Node.js installed	Install the latest LTS release.
Cannot read package.json (ENOENT)	Command run in the wrong folder	cd into the folder that contains package.json.
Port 5173 is in use	Another dev server is running	Use the new port Vite prints, or stop the other server (Ctrl+C).
Tailwind styles do not appear	Plugin, @import or CSS import missing	Check vite.config.js plugin, '@import "tailwindcss"' in style.css, and import './css/style.css' in main.js. Restart the server.
Dark mode does not change colours	Dark variant not defined	Add @custom-variant dark (&:where(.dark, .dark *)); to style.css.
Failed to resolve import './x.js'	Wrong path, name or letter case	Fix the path; include .js; match capital letters exactly.
Cannot use import statement outside a module	Script tag is not a module	Use <script type='module' src='/src/main.js'>.
Cannot read properties of null (addEventListener)	Element id in JS does not match index.html	Compare each id (employee-list, detail-panel, search...).
Failed to fetch	Offline, blocked by firewall / ad-blocker, or wrong URL	Check connection, open the URL in a new tab, disable blockers, click Retry.
HTTP 404	Wrong endpoint	Check BASE_URL in employeeApi.js.
HTTP 429	Too many requests	Wait a minute; the Session Storage cache reduces calls.
Unexpected token '<' ... is not valid JSON	Server returned an HTML page, not JSON	Open the URL and check the Network tab > Response.
users.map is not a function	API returned an error object	Inspect the response; verify the parameters.
Old data keeps showing	Cached response in Session Storage	Clear Session Storage in DevTools > Application and reload.
Favorites / theme not saved	Private mode or blocked site data	Use a normal window and allow site data.
QuotaExceededError	Storage is full	Run localStorage.clear().

Error / symptom	Likely cause	Fix
Blank page after deployment to GitHub Pages	Wrong base path	Set base: '/repo-name/' in vite.config.js and rebuild.
Vercel / Netlify build fails or 404	Wrong build settings or folder	Build command npm run build, output dir dist, correct root directory.
git push rejected	Remote has commits you do not have	git pull origin main --rebase, then git push.
Authentication failed (GitHub)	Passwords are no longer accepted	Use a Personal Access Token, GitHub Desktop or SSH.
node_modules uploaded to GitHub	.gitignore missing	Add node_modules and dist to .gitignore; git rm -r --cached node_modules.
Author identity unknown	Git user not configured	git config --global user.name / user.email.

Systematic debugging routine: (1) read the terminal output; (2) read the first red error in the Console; (3) check the Network tab for the API status; (4) check the Application tab for stored data; (5) set a breakpoint or use console.log(state); (6) search the exact error message.

14. Performance Optimisation

- **Cached API response:** Session Storage prevents repeated requests while the tab is open.
- **Debounced search:** avoids re-rendering on every key press.
- **Single-pass filtering:** one filter() call applies all conditions.
- **Lazy-loaded images:** `loading='lazy'` and fixed width / height prevent layout shift.
- **Selected fields only:** the `inc` parameter keeps the API response small.
- **Event delegation:** a few listeners instead of one per row.
- **Vite production build:** minified, bundled and tree-shaken output in the dist folder.
- **Tailwind:** only the CSS classes that are actually used are included in the build.

15. Development Process

15.1 Step-by-step build order (from the task brief)

Step	Activity
1	Install Node.js and npm.
2	Create a Vite project.
3	Learn ES6+ JavaScript features.
4	Practice asynchronous programming.
5	Consume REST APIs using Fetch.
6	Work with JSON data.
7	Implement browser storage.
8	Build employee search and filtering.
9	Handle loading and error states.
10	Test and debug the application.
11	Upload the project to GitHub.
12	Prepare documentation.

15.2 Thirty-five-day learning plan

Days	Topics
1-5	Advanced JavaScript (ES6+)
6-9	Functions, closures, array methods
10-13	Event loop, Promises, async/await
14-18	REST APIs, HTTP, Fetch API
19-21	JSON and browser storage
22-25	Node.js and npm
26-28	Vite and project structure
29-32	Employee Directory project
33-35	Testing, debugging, documentation and GitHub

15.3 Git and GitHub workflow

```
git init
git add .
git commit -m "feat: scaffold Vite project with Tailwind"
git branch -M main
git remote add origin https://github.com/your-username/your-repo-name.git
git push -u origin main

# then commit small, meaningful changes
git commit -m "feat: add API layer"
git commit -m "feat: add search and department filter"
git commit -m "feat: add favorites with Local Storage"
git commit -m "docs: add README and project documentation"
```

Commit messages use prefixes such as **feat:** (new feature), **fix:** (bug fix), **docs:** (documentation) and **style:** (visual changes), which gives the repository a professional, meaningful commit history.

16. Deployment

Platform	Settings	Notes
Vercel	Framework: Vite. Build command: npm run build. Output directory: dist.	Import the GitHub repository; every push redeploys.
Netlify	Build command: npm run build. Publish directory: dist.	Add from Git; drag-and-drop of the dist folder also works.
GitHub Pages	Set base: '/repo-name/' in vite.config.js; publish the dist folder.	Use gh-pages or a GitHub Actions workflow.

Before deploying, run `npm run build` and `npm run preview` locally to make sure the production build works.

17. Evaluation Criteria Mapping

The table shows how each marking criterion of the task is addressed by the project.

Criteria	Marks	How this project addresses it
ES6+ JavaScript	15	Modules, arrow functions, template literals, destructuring, spread / rest, default parameters, optional chaining, nullish coalescing, Set, closures.
Asynchronous Programming	15	async/await with try/catch in api and main layers; Promise-based fetch; debounce with timers; loading and error states.

Criteria	Marks	How this project addresses it
Fetch API and REST APIs	20	GET request with query parameters; response.ok check; JSON parsing; HTTP status handling; retry; API tested in Postman.
Browser Storage	10	Local Storage for favorites, theme, filter and sort; Session Storage for selection and cache; safe JSON wrapper.
Vite and Project Structure	10	Project created with Vite; professional folder structure; npm scripts; production build.
UI Responsiveness and Accessibility	10	Responsive Tailwind layout; labels, ARIA, keyboard support, focus outlines; Lighthouse audit.
Code Quality	10	Single-purpose modules, meaningful names, escaped output, consistent formatting (Prettier / ESLint).
GitHub Repository	5	Public repository, meaningful commit history, .gitignore, screenshots.
Documentation	5	README.md and this project documentation with diagrams and screenshots.
Total	**100**	

17.1 Requirement traceability (task brief to implementation)

Required feature / deliverable	Delivered by
Employee List, Search, Department Filter, Sorting	main.js (getVisible), index.html filter form, components/employeeList.js
Favorites, Local Storage	main.js (toggleFavorite), storage/storage.js
Fetch API, API integration	api/employeeApi.js, services/employeeService.js
Responsive design, Accessible UI	Tailwind breakpoints, ARIA attributes, labels, focus styles
Loading indicators, Error handling, Retry	components/stateViews.js, load() in main.js
Dark mode	Theme toggle in main.js, dark variant in style.css
Vite project, Source code, GitHub repository	Task-2 project folder and GitHub repo
Screenshots, README.md, Documentation	screenshots/ folder, README.md, this document

18. Final Submission Checklist

Done	Item
[ ]	Used modern ES6+ JavaScript features.
[ ]	Implemented asynchronous operations with Promises and async/await.
[ ]	Integrated at least one public REST API using the Fetch API.
[ ]	Parsed and managed JSON data correctly.
[ ]	Used Local Storage or Session Storage for user preferences.
[ ]	Created the project using Vite and organised it into reusable modules.
[ ]	Implemented loading, success, empty and error states.
[ ]	Tested the application in multiple browsers.
[ ]	Uploaded the complete project to GitHub with meaningful commit history.
[ ]	Added a professional README.md (installation steps, features, screenshots, folder structure, technologies, future enhancements).
[ ]	Added screenshots to the documentation and to the repository.
[ ]	Prepared this project documentation.

Deliverables: Vite project, JavaScript source code, API integration, browser storage, GitHub repository, screenshots, README.md and documentation.

19. Challenges and Key Learnings

Challenge	Solution / learning
fetch() does not reject on HTTP errors	Always check response.ok and throw an Error so the UI can show the error state.
Rows are re-created on every render, so direct listeners are lost	Use event delegation with closest() and data-attributes.
Search re-rendering too often	Debounce the input handler using a closure.
API has no departments or availability	Derive them in the service layer in a predictable way.
Storage can fail (private mode, full storage)	Wrap storage calls in try/catch and return default values.
Untrusted text inserted into HTML	Escape all API text with escapeHtml().
Keeping UI and data in sync	Single state object and one render() function.
Cannot nest a favorite button inside a row button	Make each list item contain two sibling buttons.

20. Future Enhancements and Connection to Task 3

20.1 Future enhancements

- Rebuild the application in React with routing and Context API (Task 3).
- Pagination or infinite scrolling for large directories.
- Add, edit and delete employees with a real back-end and database.
- User authentication and role-based access control.
- Export the list to CSV, and advanced filters (location, job title).
- Automated tests with Vitest and Playwright.
- Offline support with a Service Worker; CI/CD with GitHub Actions.

20.2 Connection to Task 3

Task 3 covers React.js fundamentals, JSX, components, props, state, hooks, React Router, forms, component lifecycle, Context API and Single Page Application development. It builds on the JavaScript and Vite knowledge from Task 2: the learner develops reusable components, manages application state, implements routing, handles forms and builds modern front-end applications with the React ecosystem.

Task 2 (this project)	Task 3 (React)
api/, services/, storage/, utils/ modules	Reused unchanged
components/*.js render functions	React components with props
state object and render()	useState / useReducer
load() with async/await	useEffect with loading and error state
storage.js calls	Custom useLocalStorage hook
Dark mode and favorites variables	Context API (ThemeContext, FavoritesContext)
Selected employee in Session Storage	React Router route /employee/:id
Filter inputs	Controlled form inputs; Add Employee form

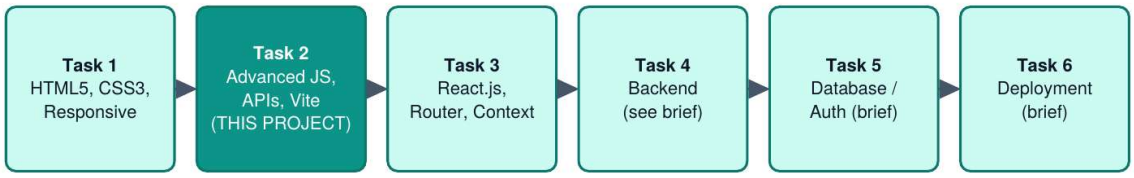


Figure 23: Internship roadmap (Tasks 1-6); Task 2 highlighted. Tasks 4-6 are described in the internship brief.

21. Conclusion

The Interactive Employee Directory fulfils the objectives of Task 2. It demonstrates modern JavaScript, asynchronous programming with Promises and async/await, REST API integration using the Fetch API, JSON handling, browser storage, npm package management and a Vite-based project with a professional modular structure. The finished application is responsive, accessible, handles loading, empty and error states, remembers user preferences, and is documented and published on GitHub.

The skills and the reusable service, API and storage layers created in this task form a strong base for React.js in Task 3 and for authentication, database integration and deployment in the capstone project.

22. References and Resources

Topic	Link
JavaScript (MDN)	https://developer.mozilla.org/en-US/docs/Web/JavaScript
Fetch API	https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
Promises	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
Async functions	https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
Web Storage	https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
HTTP	https://developer.mozilla.org/en-US/docs/Web/HTTP
JavaScript.info - Event loop	https://javascript.info/event-loop
Vite	https://vite.dev/guide/
npm	https://docs.npmjs.com/
Node.js	https://nodejs.org/en/learn
Tailwind CSS	https://tailwindcss.com/docs
Chrome DevTools	https://developer.chrome.com/docs/devtools/
Postman	https://learning.postman.com/
RandomUser API	https://randomuser.me/
JSONPlaceholder / DummyJSON / ReqRes	https://jsonplaceholder.typicode.com/ | https://dummyjson.com/ | https://reqres.in/

Appendix A. Source Code Listings

Key source files of the project. Component files that build HTML templates are described in Section 5.2 and are available in the GitHub repository.

A.1 src/js/constants.js

```
export const DEPARTMENTS = ['Engineering', 'Accounting', 'Sales', 'Marketing', 'Finance', 'Design'];

export const JOB_TITLES = {
  Engineering: ['Software Engineer', 'QA Engineer'],
  Accounting:  ['Accountant', 'Auditor'],
  Sales:       ['Sales Consultant', 'Account Manager'],
  Marketing:   ['Communication Manager', 'Marketing Coordinator'],
  Finance:     ['Financial Supervisor', 'Budget Analyst'],
  Design:      ['Product Designer', 'UX Researcher'],
};

export const STORAGE_KEYS = {
  favorites: 'ed:favorites', theme: 'ed:theme', department: 'ed:department',
  sort: 'ed:sort', selected: 'ed:selected', cache: 'ed:employees-cache',
};
```

A.2 src/utils/helpers.js

```
export function debounce(fn, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

export function escapeHtml(value = '') {
  return String(value)
    .replaceAll('&', '&amp;').replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;').replaceAll('"', '&quot;');
}
```

A.3 src/storage/storage.js

```
function makeStore(area) {
  return {
    get(key, fallback = null) {
      try {
        const raw = area.getItem(key);
        return raw === null ? fallback : JSON.parse(raw);
      } catch { return fallback; }
    },
    set(key, value) {
      try { area.setItem(key, JSON.stringify(value)); }
      catch (err) { console.warn('Storage unavailable:', err); }
    },
    remove(key) { area.removeItem(key); },
  };
}
export const local = makeStore(localStorage);
export const session = makeStore(sessionStorage);
```

A.4 src/api/employeeApi.js

```
const BASE_URL = 'https://randomuser.me/api/';

export async function fetchUsers(count = 30) {
  const params = new URLSearchParams({
    results: count, seed: 'sumerix', nat: 'us,gb,au',
    inc: 'login,name,email,phone,cell,picture,location,dob,gender',
  });
  const response = await fetch(`${BASE_URL}?${params}`);
  if (!response.ok) {
    throw new Error(`Request failed (HTTP ${response.status})`);
  }
  const data = await response.json();
  return data.results;
}
```

A.5 src/services/employeeService.js

```
import { fetchUsers } from '../api/employeeApi.js';
import { DEPARTMENTS, JOB_TITLES, STORAGE_KEYS } from '../js/constants.js';
import { session } from '../storage/storage.js';

function toEmployee(user, index) {
  const department = DEPARTMENTS[index % DEPARTMENTS.length];
  const titles = JOB_TITLES[department];
  return {
    id: user.login.uuid,
    name: `${user.name.first} ${user.name.last}`,
    title: titles[Math.floor(index / DEPARTMENTS.length) % titles.length],
    department,
    availability: index % 5 === 1 ? 'Out' : 'Available',
    phone: user.phone, mobile: user.cell, email: user.email,
    photo: user.picture.medium, office: user.location.city,
    gender: user.gender === 'male' ? 'Male' : 'Female',
    birthday: new Date(user.dob.date)
      .toLocaleDateString('en-GB', { day: 'numeric', month: 'long' }),
  };
}

export async function getEmployees() {
  const cached = session.get(STORAGE_KEYS.cache);
  if (cached) return cached;                        // optimise API requests

  const users = await fetchUsers(30);
  const employees = users.map(toEmployee);
  employees.forEach((emp) => {
    const boss = employees.find((e) => e.department === emp.department && e.id !== emp.id);
    emp.supervisor = boss ? boss.name : '-';
  });
  session.set(STORAGE_KEYS.cache, employees);
  return employees;
}
```

A.6 src/main.js (core logic)

```
const state = {
  employees: [], query: '',
  department: local.get(STORAGE_KEYS.department, 'All'),
  availability: 'All',
  sort: local.get(STORAGE_KEYS.sort, 'az'),
  favoritesOnly: false,
  favorites: new Set(local.get(STORAGE_KEYS.favorites, [])),
  selectedId: session.get(STORAGE_KEYS.selected),
};

function render() {
  const visible = getVisible();
  els.count.textContent = `${visible.length} employee${visible.length === 1 ? '' : 's'} found`;
  if (visible.length === 0) showEmpty(els.list);
  else renderList(els.list, visible, state);
  const selected = state.employees.find((e) => e.id === state.selectedId);
  renderDetail(els.detail, selected, state.favorites);
}

function toggleFavorite(id) {
  state.favorites.has(id) ? state.favorites.delete(id) : state.favorites.add(id);
  local.set(STORAGE_KEYS.favorites, [...state.favorites]);
  render();
}

els.search.addEventListener('input', debounce((e) => {
  state.query = e.target.value;
  render();
}));

// event delegation for rows, stars and the retry button
els.list.addEventListener('click', (e) => {
  const fav = e.target.closest('[data-fav]');
  const sel = e.target.closest('[data-select]');
  if (fav) return toggleFavorite(fav.dataset.fav);
  if (sel) return selectEmployee(sel.dataset.select);
  if (e.target.closest('#retry-btn')) load();
});
```

Appendix B. Interview Preparation

Suggested answers to the project questions listed in the task brief.

Question	Short answer
Explain your project architecture.	Layered: API layer (fetch), service layer (map and cache), state object in main.js, components that render HTML, plus storage and utility modules. Data flows one way into state, then render().
How did you organise JavaScript modules?	One responsibility per file in api/, services/, storage/, components/, utils/ and js/, connected with ES module import and export.
How did you handle API errors?	Checked response.ok and threw an Error, wrapped the call in try/catch, and showed an error view with a Retry button.
Why did you use async/await?	It makes asynchronous code easier to read than long .then() chains and allows normal try/catch error handling.
How did you optimise performance?	Session Storage cache, debounced search, lazy-loaded images, limited API fields, event delegation, Vite production build.
Why did you choose Vite?	Very fast dev server with hot reload, simple configuration, optimised production build, and it is the base for the React work in Task 3.
What is a closure? Where did you use one?	A function that remembers variables from its outer scope. debounce remembers timer; makeStore remembers the storage area.
Difference between Local and Session Storage?	Local Storage persists until cleared; Session Storage is cleared when the tab closes. Both store strings only.
Why does fetch() not throw on a 404?	fetch() only rejects on network failure. HTTP error statuses still resolve, so response.ok must be checked.
What is the event loop?	The mechanism that moves finished callbacks from the task and microtask queues to the call stack when the stack is empty.